Nguyên nhân gốc của lỗi patch

FOV mask đang được tính trên ảnh ĐÃ qua Ben Graham, thay vì trên ảnh gốc.

Chuỗi gọi hiện tại:

```
# Cell 20 — __getitem__
processed_np = apply_green_channel_ablation(img_rgb)   # Ben Graham
local_native_np = processed_np                         # <-- ảnh ĐÃ xử lý

# Cell 23 — collate_fn
extract_mil_patches_sliding(item["local_native_image"])
   └── compute_fov_mask(img_np)    # Otsu chạy trên ảnh Ben Graham
```

Vấn đề: Ben Graham biến nền đen thành xám 128. Công thức 4g − 4·blur + 128, ở vùng nền thì g ≈ blur ≈ 0 nên kết quả ≈ 128 — đúng bằng giá trị trung bình của võng mạc. Otsu mất hoàn toàn khả năng tách nền khỏi mô võng mạc.

Mô phỏng trên ảnh đáy mắt tổng hợp (nền đen + đĩa võng mạc tròn), chạy đúng chuỗi hàm trong code của em:

```
=== TRƯỚC Ben Graham ===
  Nền (góc ảnh)   : 0
  Võng mạc (tâm)  : 106
  Otsu -> tỷ lệ pixel được coi là FOV: 0.601    (đúng, ~0.60)

=== SAU Ben Graham (đầu vào thực tế của compute_fov_mask) ===
  Nền (góc ảnh)   : 128    <-- đen 0 đã thành XÁM 128
  Võng mạc (tâm)  : 128    <-- trùng luôn với nền
  Viền halo (biên): 0      <-- chỉ còn vành biên là tối
  Otsu -> tỷ lệ pixel được coi là FOV: 0.061    <-- SAI
```

Otsu chỉ còn nhận ra 6,1% số pixel và đám pixel đó chính là **vành halo tối ở rìa FOV** (thứ duy nhất còn tối sau Ben Graham)

Một lỗi với bốn triệu chứng

Hiện tượng em quan sát được	Giải thích
36 patch trông **giống hệt nhau**, đều là dải tối chéo	Bounding box bị co về đúng vành halo. Mọi candidate đều nằm trên cùng một vành nên nhìn như nhau.
Patch **"còn dính viền đen"**	Cái "viền đen" đó chính là halo do Ben Graham tạo ra, không phải nền gốc còn sót.
Ảnh Global có mảng trắng lớn và dải đen	addWeighted(g, 4, ...) khuếch đại 4× nên mọi lệch cục bộ vượt ±32 đều bão hoà về 0 hoặc 255.
Patch **bị trùng vị trí**	Số candidate hợp lệ tụt rất thấp → len(candidates) < m_target → np.random.choice bật replace=True → lấy lặp.

Cách sửa

Tách hai đường: FOV mask tính trên ảnh gốc, Ben Graham chỉ dùng cho nội dung ảnh.

```
# __getitem__
img_rgb  = cv2.cvtColor(cv2.imread(img_path), cv2.COLOR_BGR2RGB)
fov_mask = compute_fov_mask(img_rgb, erode_px=15)      # <-- TRÊN ẢNH GỐC
processed_np = apply_green_channel_ablation(img_rgb)

return {...,
        "local_native_image": processed_np,
        "fov_mask": fov_mask}                    # truyền mask xuống collate

# extract_mil_patches_sliding: nhận mask từ ngoài, KHÔNG tự tính lại
def extract_mil_patches_sliding(img_np, fov_mask, ...):
    ...
```

stride = 48 với patch_size = 128 là chồng lấn 62,5%, hai patch cạnh nhau dùng chung 62% pixel. Kể cả khi mask đã đúng, patch vẫn sẽ trông na ná nhau. Nâng lên stride = 96 (chồng 25%) là đủ để không bỏ sót mà vẫn đa dạng.

Khi len(candidates) < m_target, đừng lấy lặp âm thầm. Ghi logger.warning kèm native_path để biết ảnh nào có vấn đề thay vì phải đi visualize mò.

> **Cách xác nhận đã sửa đúng**
>
> Chạy lại đúng cell visualize 36 patch. Nếu các patch trở nên đa dạng và nằm rải khắp vùng võng mạc (thay vì cùng một dải tối) là em đã sửa xong. Không cần đo gì thêm.

Nút thắt tốc độ

GPU đang rảnh hơn 98% thời gian

Ước lượng tải GPU cho mỗi bước huấn luyện với cấu hình hiện tại (batch 48, M tối đa 64):

Thành phần	Khối lượng	FLOPs (forward)
Global	48 ảnh × ResNet-50 @ 512^2	≈ 1,03 TFLOPs
Local	48 × 64 = **3.072 patch** × EfficientNet-B0 @ 128^2	≈ 0,39 TFLOPs
Tổng kể cả backward		**≈ 4,2 TFLOPs / bước**

A100 với AMP đạt thực tế khoảng 100 TFLOPS → **khoảng 0,04 giây mỗi bước**. Với 146 bước một epoch, GPU chỉ cần **dưới 1 phút/epoch**. Em đang mất 90 phút. Nghĩa là nút thắt nằm hoàn toàn ở CPU và đường truyền dữ liệu, không nằm ở kiến trúc mô hình.

Thủ phạm 1, truyền 755 MB mỗi batch qua shared memory

```
patch  : 48 × 64 × 3 × 128 × 128 × float32  =  604 MB
global : 48 ×      3 × 512 × 512 × float32  =  151 MB
                              tổng  ≈  755 MB / batch

× 146 batch  ->  ~110 GB/epoch đi qua IPC worker -> main process,
                 rồi bị pin_memory copy thêm một lần nữa.
```

Đây gần như chắc chắn là chi phí lớn nhất hiện nay.

Cách sửa: trả về uint8 từ collate, chuẩn hoá trên GPU. Giảm 4 lần ngay lập tức (755 MB → 189 MB).

```
# collate: giữ uint8, không chia 255, không chuẩn hoá
patches_u8 = torch.from_numpy(patches_np).permute(0, 3, 1, 2).contiguous()

# trong training loop, sau khi .to(device):
x = patches.float().div_(255).sub_(mean_gpu).div_(std_gpu)
```

Thủ phạm 2, OpenCV tự mở thread bên trong mỗi worker

Colab A100 có khoảng 12 vCPU. Em đang chạy 8 worker, mỗi worker OpenCV mặc định lại mở thêm khoảng 12 thread → xấp xỉ 96 thread trên 12 core. Thrashing nặng có thể làm chậm 3–5 lần.

Sửa một dòng

```
cv2.setNumThreads(0)   # BẮT BUỘC khi dùng OpenCV trong DataLoader worker
                       # Đặt ở đầu notebook, ngay sau import cv2
```

Thủ phạm 3, tính lại thứ đã biết trước, 100 lần

apply_green_channel_ablation và compute_fov_mask là hàm tất định, cùng input luôn cho cùng output. Em đang tính lại chúng mỗi epoch.

Đo trên ảnh 3000×2000, một luồng CPU:

```
bilateralFilter(d=9, 75, 75)  :  187 ms
GaussianBlur(sigma=30)        :  354 ms
compute_fov_mask              :  242 ms
vòng lặp 2.400 vị trí         :   23 ms   -> 1.187 candidate
---------------------------------------------
TỔNG CPU / ảnh                : 0,81 s
```

Nhân 7.000 ảnh × 100 epoch = **khoảng 158 giờ CPU bị đốt để tính đi tính lại cùng một kết quả.**

Cách sửa: chạy một lần, lưu xuống.

```
# Chạy 1 lần với joblib.Parallel(n_jobs=12), ~10 phút cho toàn bộ dataset
np.savez_compressed(f"{CACHE}/{dataset_key}/{img_id}.npz",
                    img     = processed_u8,      # ảnh Ben Graham, uint8
                    anchors = valid_anchors)     # mảng (N,2) toạ độ patch hợp lệ
```

Chú ý: chỉ lưu TOẠ ĐỘ ANCHOR, không lưu pixel của patch. Khi đó __getitem__ chỉ còn: đọc npz → chọn ngẫu nhiên M anchor → slice. Còn khoảng 20–30 ms/ảnh thay vì 810 ms.

Một bug nữa cần sửa cùng lúc

```
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, ...,
                        collate_fn=train_collate_fn)   # <-- mode="train"!
```

Val đang dùng train_collate_fn, tức lấy ngẫu nhiên 32–64 patch mỗi lần. Hệ quả: **val QWK dao động giữa các epoch do random patch chứ không do model** nên ModelCheckpoint chọn "best val QWK" một phần là đang chọn nhiễu.

- Sửa: val_loader phải dùng eval_collate_fn.
- Dòng print bên dưới đang ghi *"Val: mode='eval'"*, sai so với code. Sửa luôn cho khớp.

Viết bài song song với thực nghiệm

Ba phần sau viết được ngay hôm nay không phụ thuộc vào bất kỳ con số kết quả nào. Thậm chí trong trường hợp mình chưa có kết quả thực nghiệm nhưng những gì mình phân tích trình bày lập luận trước đó xem như kịch bản và là story cho bài rồi nên em cứ viết song song nhé

Phần	Vì sao viết được ngay
Related Work	Em đã đọc và đánh giá nhiều bài rồi. Dùng khung phân loại 5 nhóm phương pháp ordinal ở mục 2.1–2.4 của bài Vargas làm dàn ý có sẵn.
Method	Toàn bộ kiến trúc đã cố định: dual-branch, CBAM, Non-local, Quadrant tokens, sliding-window MIL, dual pooling, CLM head. Viết được đầy đủ.
Experimental Setup	Dataset, bảng ánh xạ nhãn, chiến lược split theo patient ID, giao thức LODO, bộ metric, đã cố định hết.

Ba phần đó là khoảng 60% số chữ của bài báo, viết trong lúc chờ job chạy em nhé.